1. Selected Components

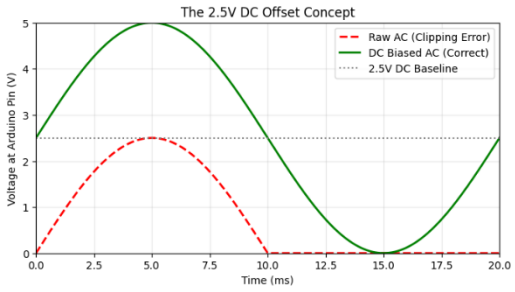
Component	Specification	Reason for Selection
Microcontroller	Arduino Uno (ATmega328P)	Provides a 10-bit ADC for accurate RMS sampling, built-in EEPROM for non-volatile threshold storage, and sufficient GPIO for UI and relay control.
Display	SSD1306 OLED (128x64, I2C)	High contrast, easily readable without a backlight. Communicates via I^2C
Current Sensor	ACS712 (Simulated via CCVS)	Hall-effect based sensor. Provides isolated current measurement with a linear voltage output (100mV/A) proportional to the AC current.
Voltage Sensor	Step-Down Transformer	Steps down the 220V AC mains grid to a safe low-voltage AC signal with a coupling factor of 0.006 for the Arduino's analog inputs.
Actuator	5V SPDT Relay	Electromechanically isolates the control logic from the high-voltage grid, allowing a 5V digital signal to safely break the 220V/10A load circuit.
User Input	Tactile Push Buttons (x3)	Configured with internal pull-up resistors. Provides a hardware interface for menu navigation without needing external resistor networks.

2. Component Integration & Circuit Architecture

The system operates as a continuous closed-loop monitor.

- **Sensing:** The Step-Down Transformer and ACS712 step down the hazardous AC grid values into safe, proportional analog signals.
- **Processing:** These signals are fed into pins A0 and A1. The Arduino runs a high-speed True RMS algorithm, sampling the AC wave for 20ms (one complete 50Hz cycle) to calculate the actual energy of the waveform.
- **Actuation:** If the calculated RMS values fall outside the EEPROM-stored parameters, the Arduino immediately drops Pin 7 to LOW, de-energizing the relay and physically disconnecting the load from the grid.
- **Feedback:** The current state, live readings, and countdown timers are continuously pushed to the SSD1306 OLED display and the Serial interface.

3. The 2.5V DC Offset Requirement



The ATmega328P ADC can only read positive voltages between 0V and 5V. Alternating Current (AC) naturally swings between positive and negative values.

- If a raw AC wave (e.g., +2.5V to -2.5V) is fed directly into the Arduino, the negative half-cycle (-2.5V) is completely clipped and ignored by the ADC, and it may permanently damage the microcontroller.
- To solve this, a voltage divider network (usually two identical resistors, like 10kΩ and 10kΩ) is used to create a steady 2.5V DC baseline at the analog pin.
- The incoming AC wave is capacitively coupled to this baseline. The +2.5V to -2.5V AC wave is effectively "lifted" to ride on top of the 2.5V DC, resulting in a safe signal that swings from 0V to 5V.
- In software, analogRead() - 512 mathematically strips this 2.5V baseline back out so the True RMS algorithm only calculates the true alternating amplitude.

4. Interaction Mechanics:

- **Navigation:** UP and DOWN buttons move the cursor (">") or increment/decrement numerical values during editing.
- **Selection (Short Press):** Pressing the MENU button briefly (<800ms) acts as an "Enter" key to dive deeper into menus, or as a "Cancel/Back" key when currently adjusting a value (reverting to the previous setting).
- **Saving (Long Press):** Holding the MENU button for ≥800ms while editing a value commits the new value to the non-volatile EEPROM and drops the user directly back to the Home Screen.

5. Serial Communication Protocol

The system allows headless configuration and monitoring via a PC terminal, useful for debugging, remote management, or automated data logging.

- **Baud Rate:** 9600 bps
- **Line Ending:** Newline (\n)

Command	Action	System Response
GET_DATA	Requests live system telemetry.	V_RMS: [val]V | I_RMS: [val]A| RELAY: ON/OFF
SET_OV=[val]	Sets Over-Voltage limit.	High Voltage set to: [val]
SET_UV=[val]	Sets Under-Voltage limit.	Low Voltage set to: [val]
SET_OC=[val]	Sets Over-Current limit.	High Current set to: [val]
SET_UC=[val]	Sets Under-Current limit.	Low Current set to: [val]